

zati

bird

有 n 只鹅，第 i 只速度为 a_i ，保证 a_i 互不相同，价值为 c_i （可能有负数），且它会在 $[t_i, t_i + L]$ 时刻出现

现在你手里有一些饲料，每当你投放一单位饲料之后此时出现的速度最快的鹅会吃掉这个饲料，提供 c_i 的收益，然后消失

你可以随意在任意时刻投放任意多的饲料，你需要最大化收益和

$$1 \leq n \leq 3 \times 10^5$$

sol - bird

把所有鹅画到二维平面上，画成 $(t_i, a_i) - (t_i + L, a_i)$ 的线段，那么每个时刻可以选择覆盖到最上面的若干条线段

我们找出所有没选的线段，一定不能有一个选的线段被没选的线段完全盖住

因此我们画出没选的线段的遮挡出的轮廓线（类似直方图），然后对这个轮廓线考虑。

我们试图在这条轮廓线上 dp，令 $f_{i,j}$ 表示轮廓线从左到右延伸到 (i, j) 。我们希望统计到所有不完全被这个轮廓线包含的线段。由于所有线段长度相等，只有以下两类：

- 左端点不在轮廓线下方的线段，这类可以在轮廓线每次向右的时候统计
- 左端点在轮廓线下方，右端点不在轮廓线下方的线段，这类可以在轮廓线向下的时候统计“穿出”轮廓线的线段。

那么我们就直接得到了一个 $O(n^2)$ 的算法，轮廓线每次只转移一步。

sol - bird

考虑优化，注意到我们的转移一定是以下几种之一：

- 向右一格
- 向上升到某个线段的左端点
- 从某个线段的右端点出发向下落

对 x 扫描线，使用数据结构维护一个 f_j 表示 (x, j) 的答案，那么三种转移分别是：

- 前缀将 f 加某个值
- 求前缀 f ，单点修改 f
- 对 $j < i$ 的每个 j ，执行 $f_j \leftarrow f_i + [j \text{ 和 } i \text{ 之间的线段价值和}]$

可以使用线段树维护，复杂度 $O(n \log n)$

night

给定一个 $1 \sim n$ 的排列，求有多少 $1 \leq i < j < k < l \leq n$ 满足 $p_i \cdot p_k = p_j \cdot p_l$ 。

$1 \leq n \leq 50000$.

sol - night

设 $p_i/p_j = x/y$, 其中 $\gcd(x, y) = 1$, 则存在 a, b 使得 p_i, p_j, p_k, p_l 可以被表示成 xa, ya, yb, xb 的形式。

$\gcd(x, y) = 1$ 可以反演掉, 那么问题转化为求有多少对 (x, y, a, b) 满足 $id_{xa} < id_{ya} < id_{yb} < id_{xb}$ 。

sol - night

注意到要么有 $x, y \leq \sqrt{n}$, 要么有 $a, b \leq \sqrt{n}$ 。

如果 $x, y \leq \sqrt{n}$, 那么枚举 x, y 之后, 每个 a/b 对应一段区间, 即求有多少 a 的区间完全在 b 左边。可以线性, 复杂度即为枚举 (x, y, a) 的复杂度, 为 $O(n\sqrt{n})$ 。

如果 $a, b \leq \sqrt{n}$, 那么枚举 a, b 之后, 每个 x/y 对应一段区间, 即求有多少 x 的区间完全包含 y 的区间。可以单 $\log$, 复杂度即为 $O(n\sqrt{n} \log n)$ 。

反演 gcd 的总复杂度为 $\sum_{1 \leq d \leq n} (n/d)^{1.5} \log(n/d) = O(n\sqrt{n} \log n)$, 可以通过。

事实上, 可以注意到要么有 $x, y \leq T$, 要么有 $a, b \leq \frac{n}{T}$, 第一部分复杂度为 $O(nT)$, 第二部分复杂度为 $O(\frac{n^2 \log n}{T})$, 可以平衡为 $O(n\sqrt{n} \log n)$ 。

lis

给定一个 $1 \sim n$ 的排列 a ，判断是否能将其划分为两个 LIS 长度相等的子序列。

$$1 \leq n \leq 2 \times 10^5$$

sol - lis

这个题尝试许多策略，使劲分讨削减无解的情况也是能做出来的，但是这不够聪明，我们不讲这个做法。

LIS 的另一种考虑方式是反链划分。设一开始的 LIS 长度为 k ，那么存在一种划分方式将原序列划分为 k 个下降序列。

并且由基本性质，如果一个序列既能被划分为 m 个下降序列，又存在一个长度为 m 的上升子序列，这个序列的 LIS 为 m 。

sol - lis

当初始序列的 LIS 长度为 $2k$ 时，找出将原序列划分为 $2k$ 条反链的方式，两个序列分别取走 k 条即可。

当初始序列的 LIS 长度为 $2k + 1$ 时，至少存在一个不在 LIS 中的元素 u ，使得 u 最后在所在子序列的 LIS 中。那么这意味着存在 u 使得包含 u 的 LIS 长度 $\geq k + 1$ 。

找到这样一个 u 之后，我们找到任意一个包含 u 的长为 $k + 1$ 的上升子序列，取走除了 u 之外的每个元素属于的反链，以及 u 本身。可以得到取走的部分以及剩下的部分 LIS 均为 $k + 1$ 。

若不存在这样的 u ，显然无解。

复杂度 $O(n \log n)$

lgs

定义长为 m 的数组 b 是好的当且仅当：

- $1 \leq b_i \leq i$
- 存在一个 $1 \sim m$ 的排列使得对每个 i , b_i 是最小的满足 $\min(p[b_i : i]) = p_i$ 的整数

给定一个长为 n 的数组 a , 求 a 最长的好子序列

$1 \leq n \leq 15000$, 要求复杂度 $O(n^2)$ 。

sol - lgs

b 是好的当且仅当所有 $[b_i, i]$ 包含且不交，证明留给读者自行思考。

这听起来很区间 dp，我们在 $[b_i, i]$ 形成的树形结构从上往下 dp，但是困难的点在于我们不知道选出来的子序列在 b 中对应的下标。

但是注意到一件事情：由于区间包含且不交，所以设我们分割出来的区间内，第一个选择的元素 a_p 一定排在 b 序列的第 a_p 个。

那么就可以设出状态了。设 $f_{l,r}$ 表示我们正在考虑区间 $[l, r]$ ，且 a_l 必须选进子序列，并且 a_l 在 b 的第 a_l 个。

sol - lgs

可以进行如下转移：

- r 没有选入子序列中， $f_{l,r} \leftarrow f_{l,r-1}$
- 整个序列没有新的分割点了， r 覆盖了整个区间：如果 $a_r = a_l$ 则
 $f_{l,r} \leftarrow f_{l,r-1} + 1$
- 枚举包含 r 的段的开头在哪里，设其为 k ，那么需要满足 $a_k = a_r$ 且
 $f_{l,k-1} \geq a_k - 1$ ，若满足则进行 $f_{l,r} \leftarrow f_{k,r-1} + 1$

只有第三种转移复杂度为 $O(n^3)$ 。注意到 $a_k = a_r$ ，随着 k 增大， $f_{l,k-1} \geq a_k - 1$ 越容易满足，而 $f_{k,r-1} + 1$ 也越来越劣（因为 a_k 不变）。

那么我们只需要维护出满足条件的第一个 k 即可。这个是容易处理的，复杂度 $O(n^2)$ 。

high

给定一个 $1 \sim n$ 的排列 a ，判断是否能将其划分为两个前缀最大值个数相等的子序列，并输出字典序最小的方案。

$$1 \leq n \leq 2 \times 10^5$$

sol - high

首先获取一些关于这个题的直觉。

注意到如果 $p = 1, 2, \dots, n$ ，那么可行当且仅当 n 为偶数，因此大概率和奇偶性有一些关系

而如果 $p_1 = n$ ，那么我们至少可以造出来一边是 1，一边是 $LIS(p[2 : n])$ 的解，剩下的点都可以被 n 挡掉

进一步地，考虑每个前缀最大值属于哪个子序列：

sol - high

【我即将在黑板上画出这张图】

当我们确定了每个前缀最大值属于哪个子序列之后，在相邻的两个红色最大值之间的红色区域，我们可以任选一个递增序列作为红色序列的前缀最大值，剩下的部分都可以用蓝色的前缀最大值挡掉。

因此如果我们考察红减蓝的值，每个红色区域可以让这个值 $+ [0 : LIS]$ ，每个蓝色区域可以让这个值 $- [0 : LIS]$ 。

不难发现同时加减没有意义，所以我们只会在红色区域/蓝色区域中的一个选。

sol - high

不妨假设我们只会在红色区域中选。那么初始假设所有前缀最大值都是蓝色，然后我们需要选择一个红色上升序列，每选到一个非前缀最大值会 $+1$ ，选到一个前缀最大值会 $+2$ ，我们需要求出是否有一个序列的权值能恰好使红减蓝为 0

而我们发现如果 x 的子序列可以选出，那么 $x - 2$ 的子序列一定也可以选出，因此我们维护奇偶性的两个树状数组

那么我们就可以用树状数组上查询即可判断是否有解，求字典序最小只要把树状数组倒着撤销判断即可，总复杂度 $O(n \log n)$

hunt

一个 $[1, n] \times [1, n]$ 的二维网格，横坐标或纵坐标为 0 或 $n + 1$ 的格子都为障碍。

网格内部额外有 m 个障碍，保证所有障碍（包括外圈的障碍）四联通。

你是一个人，你在某个空地的位置上。沿上下左右移动一格需要 2 的代价。沿对角线移动一格需要 3 的代价。

q 次询问，每次给定一个初始位置和结束位置，求是否能从初始位置到结束位置，以及最短时间。

$$1 \leq n \leq 10^5, 1 \leq m, q \leq 3 \times 10^5$$

相信大家都没听懂海峰在说什么。

sol - hunt

首先，这个障碍的拓扑结构是外面一圈障碍，内部一团空地。

因此，可以想象这个问题实际上是有一个像橡皮筋拉紧一样的最优解。

【画图】

这听起来可能有点抽象，我们用更为形式化的语言重新描述一下。

【画图】

首先，我们将原问题转化为这样一个问题：

- 现在这个网格内有一些障碍，我们需要选择一条折线，折线两个点之间的距离 $2 \max(\Delta x, \Delta y) + \min(\Delta x, \Delta y)$

sol - hunt

这两个问题的等价性还是比较明显的。

进一步地，可以发现又有一个唯一的最短折线【画图】，因为这个距离的定义满足两点之间直线最短，我们可以不断调这条折线。

那么对每次询问，我们只需要快速算出这个橡皮筋折线的路径长度即可。

sol - hunt

但是这依然不好算，因为我们无法快速维护出这个折线的拐点都在哪里。

我们再转变一下思路，注意到我们说明存在这个橡皮筋折线用到的性质只有两点之间直线最短。

因此，如果我们将距离定义为 $\Delta x, \Delta y, \Delta(x + y), \Delta(x - y)$ ，这个性质依然是满足的，最短的路线依然是我们这个橡皮筋折线。

而我们可以将 $2 \max(\Delta x, \Delta y) + \min(\Delta x, \Delta y)$ 拆成上述四个值之和，即 $\Delta x + \Delta y + (\Delta(x + y) + \Delta(x - y))/2$ ，因此我们只需要计算出这四个距离即可。

这些距离都可以看成只有一个方向上有代价。以 Δy 举例，我们将整个网格分为若干个极长空白横条【画图】，每个横条内移动无代价，横条间移动是 1 的代价，且横条形成树结构，使用任意树上求距离数据结构求解即可。复杂度 $O(n \log n)$

skate

给定一张 $n \times m$ 的字符矩形，每个字符为障碍或空地。

你现在从某个空地出发，每次可以向上下左右一个方向移动，一直移动直到撞到障碍或边界为止。

求从哪些起点出发可以经过所有空地至少一次。

$$1 \leq nm \leq 2 \times 10^6。$$

sol - skate

seriously, 我懒得想办法把这篇题解嵌入到 marp 里了。

<https://flamire.github.io/2025/07/07/task18/#23-UNR9-滑冰-K-✓>